

quiero crear una p-machine

Una **P-Machine** (máquina p-code tipo Pascal/PL-0) es básicamente una máquina virtual basada en pila.

Como ya estás trabajando en un compilador con AST y generación de código, una buena arquitectura sería:

1. **Lexer**
2. **Parser**
3. **AST**
4. **Generador de P-Code**
5. **P-Machine (VM)**

La parte de la P-Machine es la más importante: interpreta instrucciones simples como

LIT , LOD , STO , CAL , JMP , JPC , OPR , etc.

Arquitectura clásica

La máquina usa:

- pc → program counter
- sp → stack pointer
- bp → base pointer

y una pila:

</> Java



```
long[] stack = new long[10000];
```

Instrucción

Puedes definir las así:

</> Java



```
public class Instruction
{
    public OpCode op;
```

```
public int l;  
public long a;  
  
public Instruction(OpCode op, int l, long a)  
{  
    this.op = op;  
    this.l = l;  
    this.a = a;  
}  
}
```

OpCodes

</> Java



```
public enum OpCode  
{  
    LIT, // push literal  
    OPR, // operaciones  
    LOD, // load  
    STO, // store  
    CAL, // call  
    INT, // allocate  
    JMP, // jump  
    JPC // conditional jump  
}
```

Estructura de la VM

</> Java



```
public class PMachine  
{  
    public long[] stack = new long[10000];  
  
    public int pc = 0;  
    public int bp = 1;  
    public int sp = 0;  
  
    public Instruction[] code;  
  
    public PMachine(Instruction[] code)  
    {  
        this.code = code;  
    }  
}
```

```
}  
}
```

Método base()

Necesario para acceder a variables en distintos niveles léxicos.

</> Java



```
private int base(int level, int currentBp)  
{  
    int b = currentBp;  
  
    while (level > 0)  
    {  
        b = (int)stack[b];  
        level--;  
    }  
  
    return b;  
}
```

Ciclo principal

</> Java



```
public void run()  
{  
    while (true)  
    {  
        Instruction i = code[pc];  
        pc++;  
  
        switch (i.op)  
        {  
            case LIT:  
            {  
                sp++;  
                stack[sp] = i.a;  
                break;  
            }  
  
            case INT:  
            {  
                sp += i.a;  
                break;  
            }  
        }  
    }  
}
```

```

    }

    case LOD:
    {
        sp++;
        stack[sp] = stack[base(i.l, bp) + (int)i.a];
        break;
    }

    case STO:
    {
        stack[base(i.l, bp) + (int)i.a] = stack[sp];
        sp--;
        break;
    }

    case JMP:
    {
        pc = (int)i.a;
        break;
    }

    case JPC:
    {
        if (stack[sp] == 0)
            pc = (int)i.a;

        sp--;
        break;
    }

    case CAL:
    {
        stack[sp + 1] = base(i.l, bp); // static link
        stack[sp + 2] = bp;           // dynamic link
        stack[sp + 3] = pc;           // return address

        bp = sp + 1;
        pc = (int)i.a;
        break;
    }

    case OPR:
    {
        executeOperation((int)i.a);
        break;
    }
}

if (pc == 0)
    break;

```

```
}  
}
```

Operaciones OPR

La tradición PL/0 usa subcódigos:

Código	Operación
0	RET
1	NEG
2	ADD
3	SUB
4	MUL
5	DIV
6	ODD
8	EQ
9	NE
10	LT
11	GE
12	GT
13	LE

Implementación

<> Java



```
private void executeOperation(int op)  
{  
    switch (op)  
    {
```

```
case 0: // RET
{
    sp = bp - 1;
    pc = (int)stack[sp + 3];
    bp = (int)stack[sp + 2];
    break;
}

case 1: // NEG
{
    stack[sp] = -stack[sp];
    break;
}

case 2: // ADD
{
    sp--;
    stack[sp] = stack[sp] + stack[sp + 1];
    break;
}

case 3: // SUB
{
    sp--;
    stack[sp] = stack[sp] - stack[sp + 1];
    break;
}

case 4: // MUL
{
    sp--;
    stack[sp] = stack[sp] * stack[sp + 1];
    break;
}

case 5: // DIV
{
    sp--;
    stack[sp] = stack[sp] / stack[sp + 1];
    break;
}

case 8: // EQ
{
    sp--;
    stack[sp] = (stack[sp] == stack[sp + 1]) ? 1 : 0;
    break;
}

case 10: // LT
{
    sp--;
```

```

        stack[sp] = (stack[sp] < stack[sp + 1]) ? 1 : 0;
        break;
    }
}
}

```

Formato del stack

Después de un `CAL`:

Dirección	Contenido
bp+0	static link
bp+1	dynamic link
bp+2	return address
bp+3...	variables locales

Ejemplo de programa

Código equivalente a:

`</>` pascal



```
x := 2 + 3;
```

P-Code:

```

INT 0 4
LIT 0 2
LIT 0 3
OPR 0 2
STO 0 3
OPR 0 0

```



Soporte float

Como ya estabas añadiendo floats al compilador, tienes 3 opciones:

Opción 1 — todo `double`

La más fácil:

</> Java



```
double[] stack;
```

y operaciones siempre en double.

Opción 2 — tagged values

</> Java



```
class Value
{
    Type type;
    Object value;
}
```

Más flexible.

Opción 3 — NaN boxing

Más avanzada y rápida.

Recomendación

Para tu compilador actual:

Haz esto

- AST tipado
- `ValueType`
- `stack` como `Object[]`
- operaciones dinámicas

Ejemplo:

</> Java



```
Object a = stack[sp];
Object b = stack[sp + 1];
```



```

if (a instanceof Double || b instanceof Double)
{
    stack[sp] =
        ((Number)a).doubleValue() +
        ((Number)b).doubleValue();
}
else
{
    stack[sp] =
        ((Number)a).longValue() +
        ((Number)b).longValue();
}

```

Próximos pasos recomendados

1. Implementar llamadas a funciones
2. Parámetros
3. Arrays
4. Strings
5. Heap
6. Garbage collector
7. Bytecode binario
8. Debugger
9. Registers virtuales
10. JIT

Si quieres, puedo ayudarte a crear:

- ↪ una P-Machine completa en Java
- ↪ generador de bytecode
- ↪ ensamblador p-code
- ↪ debugger visual
- ↪ soporte float/string
- ↪ funciones/procedir
- ↪ closures

me parece bien pero hacemos algunos ajustes. Primero trabajamos sobre una representación de memoria que es un array de bytes.

Perfecto.

Eso cambia bastante el diseño interno y te acerca más a una VM real.

En vez de:

```
</> Java
```



```
long[] stack
```

tendrás algo como:

```
</> Java
```



```
byte[] memory
```

y la P-Machine trabajará leyendo/escribiendo valores dentro de ese bloque de memoria.

Eso te da:

- control exacto del tamaño de tipos
- soporte real para:
 - `byte`
 - `short`
 - `int`
 - `long`
 - `float`
 - `double`
 - punteros
 - structs
- compatibilidad futura con heap
- alineación
- serialización de bytecode
- ABI propia

Arquitectura recomendada

Memoria unificada

```
</> Java
```



```
public class PMachine
{
    public byte[] memory = new byte[1024 * 1024];
}
```

```
public int pc;
public int sp;
public int bp;

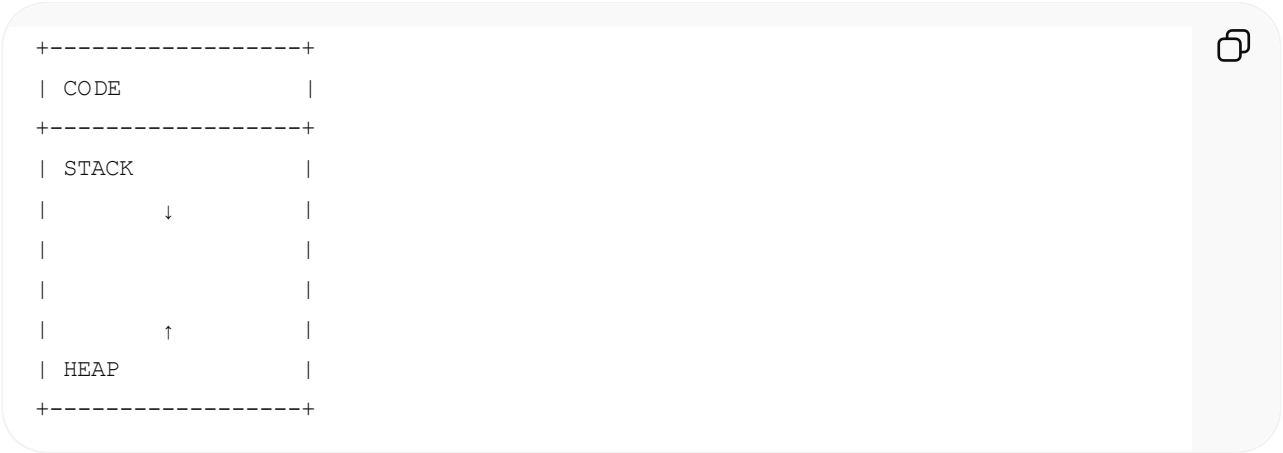
public byte[] code;
}
```

Layout típico

Puedes separar:

Región	Uso
code	bytecode
stack	stack
heap	memoria dinámica

Ejemplo:



Stack pointer

Ahora `sp` será una dirección real dentro de `memory`.

Ejemplo:

```
</> Java   
sp = 65536;
```

Lectura/escritura de tipos

Necesitas primitivas.

writeInt

</> Java



```
private void writeInt(int addr, int value)
{
    memory[addr]      = (byte) (value);
    memory[addr + 1] = (byte) (value >> 8);
    memory[addr + 2] = (byte) (value >> 16);
    memory[addr + 3] = (byte) (value >> 24);
}
```

readInt

</> Java



```
private int readInt(int addr)
{
    return
        (memory[addr] & 0xFF) |
        ((memory[addr + 1] & 0xFF) << 8) |
        ((memory[addr + 2] & 0xFF) << 16) |
        ((memory[addr + 3] & 0xFF) << 24);
}
```

Long

</> Java



```
private long readLong(int addr)
{
    long result = 0;

    for (int i = 0; i < 8; i++)
    {
        result |= ((long) (memory[addr + i] & 0xFF)) << (8 * i);
    }
}
```

```
        return result;
    }
```

Float

</> Java



```
private float readFloat(int addr)
{
    return Float.intBitsToFloat(readInt(addr));
}
```

Double

</> Java



```
private double readDouble(int addr)
{
    return Double.longBitsToDouble(readLong(addr));
}
```

Push/pop reales

pushInt

</> Java



```
private void pushInt(int value)
{
    writeInt(sp, value);
    sp += 4;
}
```

popInt

</> Java



```
private int popInt()
{

```

```
    sp -= 4;
    return readInt(sp);
}
```

MUY IMPORTANTE

Ahora la pila crece en bytes.

No en elementos.

Instrucciones tipadas

Ya no conviene:

```
LIT
ADD
MUL
```



Conviene:

```
I32_CONST
I64_CONST
F32_CONST
F64_CONST

I32_ADD
F64_ADD
```



Esto evita:

- boxing
- instanceof
- casts
- dispatch dinámico

y hace la VM muchísimo más rápida.

Diseño moderno recomendado

Bytecode basado en bytes

Ejemplo:

01 02 00 00 00



significa:

I32_CONST 2



Enum de opcodes

<> Java



```
public final class OpCode
{
    public static final byte I32_CONST = 1;
    public static final byte I32_ADD  = 2;
    public static final byte I32_SUB  = 3;
    public static final byte LOAD     = 4;
    public static final byte STORE    = 5;
    public static final byte JMP      = 6;
    public static final byte JZ       = 7;
    public static final byte CALL     = 8;
    public static final byte RET      = 9;
}
```

Fetch-decode-execute

<> Java



```
public void run()
{
    while (true)
    {
        int op = code[pc++] & 0xFF;

        switch (op)
        {
            case OpCode.I32_CONST:
            {
                int value = fetchInt();
                pushInt(value);
                break;
            }

            case OpCode.I32_ADD:
            {
```

```
        int b = popInt();
        int a = popInt();
        pushInt(a + b);
        break;
    }

    case OpCode.RET:
    {
        return;
    }
}
}
```

fetchInt

</> Java



```
private int fetchInt()
{
    int value =
        (code[pc] & 0xFF) |
        ((code[pc + 1] & 0xFF) << 8) |
        ((code[pc + 2] & 0xFF) << 16) |
        ((code[pc + 3] & 0xFF) << 24);

    pc += 4;

    return value;
}
```

Ventajas enormes de este diseño

Ya puedes implementar:

1. Structs

</> C



```
struct Vec3
{
    float x;
    float y;
```



```
float z;  
}
```

2. Arrays reales

```
base + index * size
```



3. Heap allocator

```
malloc/free
```



4. GC

mark/sweep

5. Strings UTF-8

6. Memory mapped IO

7. DLL/FFI

Recomendación MUY importante

NO uses:

```
<> Java
```

```
ByteBuffer
```



para la VM interna.

Es mucho más lento.

Haz acceso manual.

Recomendación de tipos

Usa:

Tipo VM	Tamaño
i8	1
i16	2
i32	4
i64	8
f32	4
f64	8

Recomendación para CALL

Usa stack frames reales:

```
[bp+0] return pc  
[bp+4] old bp  
[bp+8] locals...
```



Mucho más limpio.

Diseño moderno (muy recomendado)

Haz la VM estilo:

- WASM
- Lua VM
- JVM
- CLR

NO estilo PL/0 antiguo.

PL/0 sirve para aprender pero escala mal.

Siguiente paso lógico

Ahora deberíamos diseñar:

1. formato exacto de bytecode
2. encoding de instrucciones
3. layout del stack frame
4. sistema de tipos
5. calling convention
6. heap allocator
7. registros virtuales o stack machine
8. constantes
9. strings

10. loader



la pila se mueve de 4 bytes en 4 bytes. Los enteros tipo `i8` o `i16` se almacenan en el espacio que corresponde pero cuando hacemos cálculos utilizamos `i32` como en Java

Eso es un diseño muy bueno.

De hecho se parece bastante al modelo de la JVM y también a muchas VMs modernas.

La idea sería:

- **la unidad mínima del stack es 4 bytes**
- todos los operandos ocupan al menos un slot de 32 bits
- `i8` e `i16` se almacenan dentro de esos 4 bytes
- las operaciones aritméticas trabajan sobre `i32`

Exactamente como hace Java con:

- `byte`
- `short`
- `char`

que internamente se promocionan a `int`.

Diseño del stack

Cada slot = 4 bytes

```
sp -> +---+---+---+---+
      |           i32      |
      +---+---+---+---+
```



Entonces:

Tipo	Tamaño lógico	Tamaño en stack
i8	1	4
i16	2	4
i32	4	4
f32	4	4
i64	8	8
f64	8	8

Ventajas enormes

1. Stack muchísimo más simple

El SP siempre avanza:

```
</> Java
```



```
sp += 4;
```

O:

```
</> Java
```



```
sp += 8;
```

Nunca 1 ni 2.

2. Alineación perfecta

Todo queda alineado a 4 bytes.

Muy importante para rendimiento.

3. Operaciones simples

El ALU virtual siempre trabaja en 32 bits:

</> Java



```
int a = popInt();  
int b = popInt();  
  
pushInt(a + b);
```

Ejemplo i8

push byte

</> Java



```
private void pushByte(byte value)  
{  
    writeInt(sp, value);  
    sp += 4;  
}
```

Aquí el `byte` queda sign-extended automáticamente.

Ejemplo:

FF FF FF FE



para `-2`.

i16

</> Java



```
private void pushShort(short value)  
{  
    writeInt(sp, value);
```

```
    sp += 4;
}
```

Lectura

</> Java



```
private int popInt()
{
    sp -= 4;
    return readInt(sp);
}
```

Siempre devuelve `int`.

Conversión implícita

Entonces:

```
i8 + i8
```



realmente hace:

```
i32 + i32
```



Perfecto.

Instrucciones

Ahora tu ISA puede simplificarse muchísimo.

En vez de

```
I8_ADD
I16_ADD
I32_ADD
```



solo necesitas:



Conversiones

Solo necesitas:

```
I2B
I2S
I2L
I2F
I2D
```



igual que JVM.

Ejemplo I2B

</> Java



```
case I2B:
{
    int v = popInt();
    pushInt((byte)v);
    break;
}
```

Comparaciones

También funcionan sobre `i32`.

</> Java



```
case I32_LT:
{
    int b = popInt();
    int a = popInt();

    pushInt(a < b ? 1 : 0);
    break;
}
```

Variables locales

También deberían ocupar slots de 4 bytes.

Layout

```
bp+0    return address
bp+4    old bp
bp+8    local0
bp+12   local1
bp+16   local2
```



i64 y f64

Estos ocupan 2 slots.

Ejemplo

```
sp ->
+---+---+---+---+
| low 32 bits      |
+---+---+---+---+
| high 32 bits     |
+---+---+---+---+
```



pushLong

</> Java



```
private void pushLong(long value)
{
    writeInt(sp, (int) value);
    writeInt(sp + 4, (int) (value >> 32));
    sp += 8;
}
```

popLong

</> Java



```
private long popLong()
{
    sp -= 8;

    long low  = readInt(sp) & 0xFFFFFFFFL;
    long high = readInt(sp + 4) & 0xFFFFFFFFL;

    return low | (high << 32);
}
```

Recomendación MUY importante

Haz el stack:

</> Java



```
byte[] memory
```

pero:

TODAS las operaciones internas en `int`

excepto:

- long
- double

Eso simplifica enormemente el compilador.

Recomendación para bytecode

Haz instrucciones de 1 byte:

```
opcode operand operand operand
```



Ejemplo

```
01 05 00 00 00
```



=

ICONST 5



ISA recomendada

Constantes

ICONST

LCONST

FCONST

DCONST



Load/store

ILOAD

ISTORE

LLOAD

LSTORE



Operaciones

IADD

ISUB

IMUL

IDIV



Comparación

ICMPEQ

ICMPLT



Control

JMP

JZ



Esto es importante

Tu VM ya NO es una "P-Machine clásica".

Ahora estás construyendo algo mucho más parecido a:

- JVM
- CLR
- Lua VM
- WASM stack machine



Y eso es bueno.

vale. Vayamos al fichero del código. Tendremos una cabecera, un espacio para constantes y el código. Cuando se cargue en memoria tendremos la cabecera, las constantes, las variables estáticas, el código, el stack y el heap.

Sí.

Ese diseño ya es el de una VM seria y escalable. Estás definiendo un verdadero formato ejecutable para tu máquina virtual.

La separación:

- cabecera
- pool de constantes
- datos estáticos
- código
- stack
- heap

es exactamente lo correcto.

Diseño general del ejecutable

Archivo bytecode



Un único bloque

</> Java



```
byte[] memory;
```

y offsets.

Muchísimo más eficiente.

Layout recomendado

Ejemplo:

```
0x00000000 HEADER
0x00000100 CONSTANTS
0x00001000 STATICS
0x00002000 CODE
0x00010000 STACK
0x000F0000 HEAP
```



Cabecera

La cabecera debería contener:

Campo	Tamaño
magic	4
version	2
flags	2
constantPoolOffset	4
constantPoolSize	4
staticSize	4
codeOffset	4
codeSize	4
entryPoint	4

Magic number

Muy importante.

Ejemplo:

PVM1



Ejemplo binario

50 56 4D 31



Clase Header

</> Java



```
public class ExecutableHeader
{
    int magic;
    short version;
    short flags;

    int constantPoolOffset;
    int constantPoolSize;

    int staticSize;

    int codeOffset;
    int codeSize;

    int entryPoint;
}
```

Constant Pool

Muy importante.

NO pongas strings o números grandes inline en el código.

Constantes posibles

Tipo	Descripción
INT	i32
LONG	i64
FLOAT	f32
DOUBLE	f64
STRING	utf8
FUNCTION	referencia
CLASS	futuro
FIELD	futuro

Ejemplo

```
CONST #0 = "Hola"  
CONST #1 = 1234  
CONST #2 = 3.14
```



Bytecode

Entonces:

```
LOAD_CONST 0  
PRINT
```



Ventajas enormes

1. Código más pequeño

2. Strings deduplicados

3. Carga rápida

4. Más cache-friendly

Variables estáticas

Muy buena decisión separarlas.

Variables globales

```
static int x;  
static float y;
```



van aquí.

Recomendación

Reserva directamente:

```
staticSize
```



bytes.

Código

Aquí va el bytecode puro.

IMPORTANTE

Usa:

PC relativo al inicio del código

NO absoluto.

Entonces:

</> Java



```
pc = codeBase + entryPoint;
```

Stack

Tu stack debería crecer:

hacia abajo

como CPUs reales.

Ejemplo

STACK TOP



Heap

El heap creciendo hacia arriba.

Entonces

STACK ↓

...

HEAP ↑



Muy importante

Cuando:

```
heap >= stack
```



→ stack overflow / out of memory.

Recomendación para el stack frame

Layout

```
bp+0    old bp
bp+4    return address
bp+8    local0
bp+12   local1
```



CALL

Hace:

1. push old bp
 2. push return pc
 3. bp = sp
 4. jump
-

RET

Hace:

1. restore sp
 2. restore bp
 3. restore pc
-

Recomendación importante

Define ya:

Endianness

Usa:

little endian

como x86/JVM/WASM.

Recomendación MUY importante

NO mezcles:

- direcciones físicas
 - offsets de bytecode
-

Haz:

En el archivo

todo relativo.

En runtime

todo absoluto.

Ejemplo

Archivo:

```
CALL 120
```



Runtime:

```
pc = codeBase + 120
```



Recomendación para instrucciones

Hazlas de tamaño variable.

Ejemplo

Sin operandos

IADD



1 byte.

Con operandos

ICONST



5 bytes.

Ejemplo real

01 0A000000

02



=

ICONST 10

IADD



Muy importante

Diseña desde ya:

ABI

Convención de llamada

Define:

Tema

Decisión

quién limpia stack

caller/callee

paso parámetros

stack

retorno

stack

alignment

4

Tema	Decisión
64-bit alignment	8 o 4

Recomendación

Haz:

caller pushes args

```
argN  
...  
arg0  
CALL
```



retorno

push en stack.

Esto te permite luego

- funciones nativas
 - FFI
 - JIT
 - DLLs
 - syscalls
-

Siguiente paso lógico

Ahora deberíamos definir:

1. formato binario exacto
2. constant pool encoding
3. ISA completa
4. stack frame exacto
5. ABI
6. loader
7. assembler

vale. a ver la máquina virtual se ejecutaría en un procesador ARM de 32 bits o similar. El formato binario el que sea mas eficiente.

Entonces hay que diseñarla pensando como una VM “nativa” para ARM32, no como una VM académica.

Eso cambia varias decisiones importantes.

Si el objetivo es:

- ARM 32-bit
- máxima eficiencia
- poca RAM
- buen rendimiento
- posible embedded/OS propio/consola/etc.

entonces conviene acercarse más a:

- LuaJIT VM
- Dalvik
- WASM runtime
- UCSD p-System moderno

y menos a JVM clásica.

Decisión fundamental

Tienes dos opciones:

Modelo	Ventaja	Desventaja
Stack machine	bytecode compacto	más operaciones push/pop
Register VM	más rápida	bytecode más grande

Para ARM32 recomiendo

Stack machine híbrida

Como:

- WASM
- Lua
- Forth moderno

porque:

- el bytecode es mucho más pequeño
- mejor cache locality
- menos RAM
- loader simple
- compilador más sencillo

y ARM32 suele sufrir más por memoria que por ALU.

Formato binario recomendado

Todo little-endian

ARM moderno usa little-endian normalmente.

Estructura recomendada

```
+-----+
| HEADER |
+-----+
| CONSTANT POOL |
+-----+
| STATIC DATA |
+-----+
| CODE |
+-----+
```



Header recomendado

NO uses estructuras Java-style.

Hazlo plano y alineado.

Tamaño

32 bytes exactos

Muy cache-friendly.

Header propuesto

Offset	Tamaño	Campo
0	4	magic
4	2	version
6	2	flags
8	4	constOffset
12	4	constSize
16	4	staticSize
20	4	codeOffset
24	4	codeSize
28	4	entryPoint

Magic

PVM1



Importante

Todo alineado a 4 bytes.

SIEMPRE.

Constant pool

Aquí hay una decisión crítica.

Opción 1 — tagged pool

```
[type] [data]
```



Flexible pero más lenta.

Opción 2 — pools separadas

```
int pool  
float pool  
string pool
```



Más rápida.

Para ARM32 recomiendo

tagged pool simple

porque:

- loader más simple
 - compilador más simple
 - tamaño menor
-

Formato recomendado

```
u8 type  
u32 size  
data...  
padding
```



Ejemplo string

```
01
00000005
Hello
00 00 00
```



Código

Aquí está la parte importante.

Recomendación fuerte

instrucciones de 1 byte

como opcodes.

Ejemplo

```
01
02
03
```



Operand encoding

Usa:

immediate little-endian inline

Ejemplo

```
ICONST 5
```



=

```
01 05 00 00 00
```



Esto es MUY eficiente en ARM

porque puedes hacer:

```
</> C
```

```
*(int*) (pc)
```



casi directamente.

Muy importante

evita instrucciones de tamaño fijo

En ARM32 importa MUCHO el tamaño del bytecode.

Ejemplo

Bueno

```
IADD
```



1 byte.

Malo

```
IADD 00 00 00
```



4 bytes desperdiciados.

Instrucciones recomendadas

Stack

```
PUSH_I32
```

```
PUSH_CONST
```



POP
DUP

Locales

LOAD_LOCAL
STORE_LOCAL



Globales

LOAD_STATIC
STORE_STATIC



Arithmetic

IADD
ISUB
IMUL
IDIV



Comparación

ICMPEQ
ICMPLT



Control

JMP
JZ
CALL
RET



Memoria

```
LOAD8  
LOAD16  
LOAD32  
  
STORE8  
STORE16  
STORE32
```



MUY importantes.

Esto permite

- arrays
 - structs
 - objetos
 - punteros
 - heap
 - MMIO
-

Stack frame recomendado

Muy parecido a ARM real

```
[args]  
[return pc]  
[old bp]  
[locals]
```



SP

Crece hacia abajo.

CALL

```
push return pc  
push old bp
```



```
bp = sp  
pc = target
```

RET

```
sp = bp  
restore bp  
restore pc
```

Recomendación MUY importante

Usa direct threading luego

NO switch-case.

Al principio

haz:

```
</> Java  
  
switch (op)
```

Más adelante

computed goto/direct threaded interpreter.

En ARM mejora muchísimo.

Otra recomendación MUY importante

Haz:

bytecode verifiable

Por ejemplo

Prohibido:

- stack underflow
 - saltos inválidos
 - code execution outside code section
-

Verifier

Como JVM/WASM.

Esto evita crashes.

Heap

Recomendación:

bump allocator inicial

```
heapPtr += size
```



Mucho más adelante

- free lists
 - generational GC
 - compacting GC
-

Strings

Usa:

UTF-8

NO UTF-16.

En ARM32 la RAM importa muchísimo.

ABI recomendada

caller cleanup

Más flexible.

Retorno

Siempre en stack.

Tipos recomendados

Tipo	Tamaño
i8	1
i16	2
i32	4
i64	8
f32	4
f64	8
ptr	4

MUY importante

Haz punteros explícitos.

```
PTR_ADD  
PTR_SUB  
LOAD_PTR
```



Porque luego podrás tener

- structs
- arrays
- objetos

- vtables
 - FFI
-

Mi recomendación final

Arquitectura ideal para tu VM

Runtime

- stack machine
- stack slots de 4 bytes
- byte[] memory
- little endian

Binary

- header plano 32 bytes
- constant pool tagged
- variable-length instructions

Execution

- threaded interpreter
- stack frames ARM-like

Memory

- stack ↓
- heap ↑

Types

- i32 como tipo ALU base

Future-proof

- punteros explícitos
- memory ops
- verifier

Ese diseño puede evolucionar perfectamente hacia:

- JIT ARM
- AOT compiler
- OS runtime
- scripting language...
- game VM

perfecto, guardalo todo y seguimos mas adelante.

Memoria actualizada

Perfecto, queda guardado. Continuaremos desde este diseño de la VM ARM32 la próxima vez.

